

Length & Algorithmic Generalization

positional extrapolation, simple programs, identifiability, quantitative training-length bounds

이번 주의 질문은 단순히 “Transformer가 긴 입력을 처리할 수 있는가?”가 아니다. 훈련에서는 길이 $n \leq N$ 만 보았는데, 왜 어떤 문제에서는 $n \gg N$ 에서도 같은 알고리즘을 실행하고, 다른 문제에서는 아주 조금만 길어져도 무너지는가를 묻는다. 핵심은 세 층으로 나뉜다. 첫째, 위치 표현이 길이 바깥에서 어떤 계산을 허용하는가. 둘째, 짧은 데이터와 일치하는 수많은 함수 중에서 왜 길이 전체에 통하는 같은 알고리즘을 선택하는가. 셋째, “충분히 길게 학습하면 언젠가 된다”는 정성적 명제를 실제 필요한 훈련 길이의 상계로 바꿀 수 있는가. 이 노트는 Kazemnejad et al.의 positional-encoding 실험에서 출발해 Zhou et al.의 RASP-L 가설, Huang et al.의 limit-transformer identifiability, Izzo et al.의 2026 quantitative bound까지 한 흐름으로 연결한다.

Primary readings

Amirhossein Kazemnejad et al., **The Impact of Positional Encoding on Length Generalization in Transformers**, NeurIPS 2023.

Hattie Zhou et al., **What Algorithms Can Transformers Learn? A Study in Length Generalization**, ICLR 2024.

Xinting Huang et al., **A Formal Framework for Understanding Length Generalization in Transformers**, ICLR 2025.

Zachary Izzo, Eshaan Nichani, and Jason D. Lee, **Quantitative Bounds for Length Generalization in Transformers**, ICLR 2026.

Kaiying Hou et al., **Universal Length Generalization with Turing Programs**, ICML 2025.

Contents

1	문제 설정과 기호	3
1.1	Length generalization의 정확한 문제	3
1.2	논문별 기호 대응표	3
1.3	세 질문을 분리해야 한다	4
2	Positional Encoding과 길이 외삽	5
2.1	왜 위치 표현이 먼저 등장하는가	5
2.2	Kazemnejad et al.: NoPE가 위치를 모르는 것은 아니다	5
2.3	PE가 외삽 가능하면 length generalization도 되는가?	7
3	Zhou et al.: RASP-L과 단순 알고리즘 가설	8
3.1	RASP를 Transformer의 assembly language로 보기	8
3.2	RASP-L의 제한	8
3.3	RASP-Generalization Conjecture	8
3.4	Scratchpad는 왜 어떤 때는 돕고 어떤 때는 방해하는가	9
3.5	RASP-L 관점의 장점과 한계	9
4	Huang et al.: 식별 가능성을 정리로 만들기	10
4.1	RASP-L conjecture에서 identifiability theorem으로	10
4.2	Limit Transformer	10
4.3	Idealized inference procedure	11
4.4	Theorem 7: eventual length generalization	11
4.5	왜 bounded regularizer가 중요한가	11
4.6	C-RASP와 lower/upper characterization	12
5	Izzo et al.: “충분히 길게”는 얼마나 긴가	13
5.1	Huang의 theorem이 남긴 정량적 질문	13
5.2	Finite precision에서 softmax가 hardmax가 되는 이유	13
5.3	Theorem 4.1: finite-precision, one-layer bound	14
5.4	왜 empirical frequency를 보존하면 되는가	14
5.5	Infinite precision: soft attention을 유지하면	15
5.6	Huang과 Izzo를 한 문장으로 연결하면	15
6	Algorithmic generalization을 동역학으로 보기	16
6.1	같은 transition rule을 더 오래 반복한다	16
6.2	Local step error가 길이에 따라 어떻게 누적되는가	16
6.3	Scratchpad와 locality	17
6.4	Fixed depth와 autoregressive depth의 차이	17
7	Turing Programs: 모든 알고리즘에 같은 형식을 줄 수 있는가	18
7.1	RASP-L simplicity의 task dependence	18
7.2	왜 이 형식이 length generalization에 유리한가	18
7.3	Week 7의 CoT와 연결	18
8	통합 관점: 무엇이 길이 일반화를 결정하는가	20

8.1	네 논문의 역할을 한 줄씩	20
8.2	Length generalization의 네 조건	20
8.3	서로 모순처럼 보이는 결과를 정리하기	21
8.4	발표에서 꼭 구분할 네 단어	21
9	발표용 정리와 예상 질문	22
9.1	칠판에 남겨야 할 핵심 식	22
9.2	예상 질문 1: “그러면 NoPE가 가장 좋은 PE인가?”	22
9.3	예상 질문 2: “RASP-L conjecture는 증명된 것인가?”	23
9.4	예상 질문 3: “Theorem 7이면 충분히 오래 학습하면 항상 일반화하는 것 아닌가?”	23
9.5	예상 질문 4: “Izzo bound는 necessary condition인가?”	23
9.6	예상 질문 5: “RoPE나 ALiBi가 있으면 무한 길이에 외삽 가능한가?”	23
9.7	예상 질문 6: “Turing Programs와 Turing completeness는 같은 말인가?”	23
9.8	마지막 3분 요약	23
	참고문헌	25

1

문제 설정과 기호

1.1 Length generalization의 정확한 문제

길이 일반화(length generalization)는 훈련에서 보지 못한 더 긴 입력에 대한 외삽이다. 알파벳을 Σ 라 하고, 목표 함수를

$$f : \Sigma^* \rightarrow \mathcal{Y}$$

라 쓰자. 최대 훈련 길이가 N_{tr} 이면 학습기는

$$\{(x, f(x)) : |x| \leq N_{\text{tr}}\}$$

에서 정보를 얻지만, 시험에서는

$$|x| > N_{\text{tr}}$$

인 입력을 맞춰야 한다.

보통의 i.i.d. generalization과 다른 점은 test distribution이 단순히 같은 입력 공간에서 표본만 바뀌는 것이 아니라, 입력 차원 자체가 증가한다는 점이다. 따라서 “훈련 loss가 작다”만으로는 긴 길이의 계산 규칙을 고정할 수 없다.

핵심 Length generalization에서 어려운 것은 parameter 수가 부족해서가 아니다. 짧은 길이에서 완전히 동일하게 보이는 여러 알고리즘 중 어떤 것이 긴 길이에서도 맞는지를 짧은 데이터만 보고 식별해야 한다는 것이 핵심이다.

1.2 논문별 기호 대응표

개념	이 노트	Huang et al.	Izzo et al.
입력 문자열	$x \in \Sigma^*$	$x \in S \subset \Sigma^*$	$x \in \Sigma^*$
길이	$n = x $	$ x $, context n	$ x $, training threshold N
주기	Δ	PERIODIC의 period Δ	Δ
국소 범위	τ	LOCAL의 locality radius	τ
위치 상호작용	$\phi(j, i)$	$\phi_{l,h}(j, i)$	$\phi_{l,h}(j, i)$
정규화 복잡도	$\mathcal{R}(T)$	norm-based regularizer $\mathcal{R}$	weight complexity L 또는 $C(f)$
정밀도	p bits	fixed fractional precision	finite-precision p / infinite precision
attention gap	γ	직접 핵심 기호는 아님	logit/positional margin γ
오차	ε	exact identification 중심	$\ f - g\ \leq \varepsilon$

Zhou et al.은 다른 표기 체계를 쓴다. 그 논문의 핵심 대상은 숫자 상계보다 “짧은 RASP-L 프로그램”이다. 즉 Huang/Izzo가 어떤 함수가 식별되는가와 얼마나 긴 데이터가 필요한가를

정량화한다면, Zhou et al.은 학습기의 inductive bias를 프로그램 길이 관점에서 설명하는 가설을 제시한다.

1.3 세 질문을 분리해야 한다

길이 일반화 논의를 읽을 때 다음 세 명제를 섞으면 안 된다.

1. **Representability:** 하나의 고정된 Transformer가 모든 길이에서 목표 알고리즘을 표현할 수 있는가?
2. **Identifiability:** 짧은 길이의 관측만으로 그 알고리즘을 다른 후보들과 구별할 수 있는가?
3. **Learnability:** 실제 SGD가 그 후보를 선택하는가?

Zhou et al.의 RASP-L conjecture는 세 번째에 대한 phenomenological conjecture이다. Huang et al.은 특정 idealized inference procedure 아래 두 번째를 정리로 만든다. Izzo et al.은 Huang의 “언젠가”를 훈련 길이 N 의 quantitative bound로 바꾼다.

보충 유도 — 논문 밖의 독자용 전개 훈련 길이 N 만 보고는 아무 가정 없이 길이 일반화를 보장할 수 없다. 실제로 임의의 목표 f 와 임의의 다른 함수 g 에 대해

$$\tilde{f}_N(x) = \begin{cases} f(x), & |x| \leq N, \\ g(x), & |x| > N \end{cases}$$

를 정의하자. 그러면 훈련 길이 이하에서는

$$\tilde{f}_N(x) = f(x)$$

이므로 어떤 학습 데이터도 둘을 구별하지 못한다. 그런데 g 를 f 와 다르게 택하면 긴 입력에서는 완전히 다른 답을 낸다. 따라서 length generalization theorem에는 반드시 hypothesis-class restriction, simplicity bias, locality/periodicity 같은 구조적 가정이 들어 가야 한다.

발표에서 이렇게 말하기 “길이 일반화는 단순한 i.i.d. generalization이 아닙니다. 짧은 길이에서는 동일하게 보이지만 긴 길이에서 갈라지는 알고리즘이 무한히 많기 때문에, 어떤 형태의 inductive bias나 hypothesis restriction 없이는 원리적으로 보장할 수 없습니다.”

2

Positional Encoding과 길이 외삽

2.1 왜 위치 표현이 먼저 등장하는가

Transformer의 attention logit을 단순화해서

$$s_{ij} = q_i^\top k_j + b(i, j)$$

라고 쓰자. 여기서 $b(i, j)$ 가 위치 정보를 넣는 방식이다. 훈련 길이 밖에서 $b(i, j)$ 가 어떻게 정의되고, 그 값의 scale이 어떻게 변하는지는 긴 입력에서 attention pattern을 직접 바꾼다.

대표적인 네 형태는 다음과 같다.

Absolute positional embedding (APE).

$$h_i^{(0)} = E(x_i) + p_i.$$

학습된 p_i 가 훈련 중 등장하지 않은 index에 대해 정의되지 않거나 제대로 학습되지 않을 수 있다.

T5-style relative bias.

$$s_{ij} = q_i^\top k_j + b_{\text{bucket}(i-j)}.$$

절대 위치보다 상대 거리의 bucket을 사용한다.

ALiBi.

$$s_{ij} = q_i^\top k_j - m_h(i - j).$$

거리에 따라 선형 bias를 주므로 정의 자체는 임의 길이로 외삽된다.

RoPE. query와 key를 위치별 회전하면

$$\langle R_{i\theta} q_i, R_{j\theta} k_j \rangle = q_i^\top R_{(j-i)\theta} k_j.$$

따라서 inner product는 상대 displacement를 반영한다.

2.2 Kazemnejad et al.: NoPE가 위치를 모르는 것은 아니다

Kazemnejad et al.은 decoder-only Transformer에서 APE, T5 relative PE, ALiBi, RoPE, NoPE를 비교한다. 그들의 실험에서는 여러 reasoning/mathematical synthetic task에서 NoPE가 강한 length generalization을 보였다. 중요한 점은 이것을 “NoPE가 항상 최고”라는 보편 정리로 읽으면 안 된다는 것이다. 논문의 실험 범위와 optimization setting에서의 관찰이다.

더 흥미로운 결과는 **causal mask** 자체가 위치 정보를 제공할 수 있다는 구성이다.

NoPE로 absolute index를 복원하는 구성

원 논문 기준 Kazemnejad et al.의 이론적 구성은 고정된 최대 길이까지 NoPE Transformer가 absolute/relative positional encoding을 표현할 수 있음을 보인다. 이 자체가 “하나의 고정 폭 모델이 모든 길이로 일반화한다”는 정리는 아니다.

입력이

$$[\text{BOS}, x_1, \dots, x_T]$$

라고 하자. 모든 key의 logit이 동일하도록 한 attention head를 구성하면 causal mask 때문에 위치 t 에서 볼 수 있는 t 개의 token에 대해

$$\alpha_{tj} = \frac{1}{t}, \quad j \leq t$$

가 된다.

value는 BOS에서만 1, 나머지에서는 0이 되도록 하면 attention output은

$$\sum_{j \leq t} \alpha_{tj} v_j = \frac{1}{t}.$$

즉 causal prefix size가 $1/t$ 라는 scalar에 인코딩된다.

최대 길이가 $T_{\max}$ 로 고정되어 있으면 입력 집합

$$\left\{ 1, \frac{1}{2}, \dots, \frac{1}{T_{\max} + 1} \right\}$$

은 유한하다. 충분히 넓은 ReLU MLP는 이 유한 집합 위에서

$$\frac{1}{t} \mapsto t$$

를 정확히 구현할 수 있다. 따라서 explicit PE가 없어도 absolute position을 표현할 수 있다.

주의 이 결과에서 MLP가 처리해야 하는 점의 개수는 최대 context와 함께 증가한다. 따라서 “각 유한 $T_{\max}$ 마다 구현 가능”과 “폭/복잡도를 고정한 한 모델이 모든 T 에 대해 같은 계산을 한다”는 서로 다른 명제다. Huang et al.의 framework에서는 바로 이 차이가 regularizer의 boundedness로 드러난다.

NoPE로 relative position을 만들기

absolute index t 가 내부 표현에 저장되어 있다고 하자. query/key의 일부 좌표를

$$q_t = (1, -t, \dots)^\top, \quad k_i = (i, 1, \dots)^\top$$

로 잡으면

$$q_t^\top k_i = i - t + \sum_{r \geq 3} q_{t,r} k_{i,r}.$$

따라서 attention score를

$$\underbrace{i - t}_{\text{relative position}} + \underbrace{f_{\text{content}}(x_t, x_i)}_{\text{content term}}$$

꼴로 만들 수 있다. 이 구성은 NoPE가 “position-blind” 모델이라는 단순한 해석이 틀렸음을 보여 준다.

2.3 PE가 외삽 가능하면 length generalization도 되는가?

그렇지 않다. RoPE/ALiBi 같은 함수가 임의 index에서 정의된다는 것은 위치 함수의 extrapolation이고, 목표 알고리즘이 긴 길이에서도 유지된다는 것은 computational extrapolation이다. 두 조건은 다르다.

보충 유도 — 논문 밖의 독자용 전개 attention dilution을 단순 모델로 보자. 관련 token 하나의 logit이 모든 배경 token보다 Δ 만큼 크고, 배경 token이 $N-1$ 개 있다고 하자. 관련 token의 softmax weight는

$$\alpha_* = \frac{e^\Delta}{e^\Delta + N - 1} \leq \frac{e^\Delta}{N - 1}.$$

$\Delta = O(1)$ 이면

$$\alpha_* = O(N^{-1}).$$

따라서 길이가 커질수록 특정 local signal의 mass가 희석될 수 있다. 이를 상수 수준으로 유지하려면 대략

$$\Delta \gtrsim \log N$$

크기의 logit separation이 필요하다. 이 계산은 특정 논문의 정리가 아니라, Huang/Izzo가 왜 $\log|x|$ scaling 또는 locality를 명시적으로 다루는지 이해하기 위한 보충 유도다.

발표에서 이렇게 말하기 “Positional encoding이 긴 index에 정의된다는 것만으로는 충분하지 않습니다. 더 근본적인 질문은 같은 attention computation과 같은 알고리즘이 길이가 커져도 유지되는가입니다. NoPE조차 causal mask로 위치를 복원할 수 있으므로, PE 이름만 보고 length generalization을 판정할 수 없습니다.”

3

Zhou et al.: RASP-L과 단순 알고리즘 가설

3.1 RASP를 Transformer의 assembly language로 보기

RASP는 sequence-to-sequence 계산을 Transformer 구조에 가까운 primitive로 기술하는 언어다. Zhou et al.이 사용하는 causal RASP의 핵심 연산은 대략

- elementwise map,
- 두 sequence를 elementwise 결합하는 seq_map,
- causal attention에 대응하는 kqv

이다. 프로그램은 **straight-line program**이고, 임의 횟수의 loop나 일반적인 control flow는 허용하지 않는다. 각 RASP line이 대략 한 Transformer layer로 compile된다는 점 때문에 프로그램 길이가 Transformer-native한 simplicity measure가 된다.

3.2 RASP-L의 제한

원래 RASP는 표현은 가능하지만 실제 학습이 불안정한 연산까지 허용할 수 있다. Zhou et al.은 이를 줄인 learnable fragment를 RASP-L이라 부른다. 핵심 제한은 다음과 같다.

- 변수는 bounded integer 범위에서 다룬다.
- token index는 특별하게 취급한다.
- index에 대해서 order comparison, predecessor, successor 같은 단순 연산은 허용하지만 임의 arithmetic을 제한한다.
- 전체 계산은 고정 길이 straight-line program이어야 한다.

이 제한은 “무엇이 수학적으로 표현 가능한가”보다 “Transformer가 어떤 uniform algorithm을 쉽게 학습하는가”를 모델링하려는 선택이다.

3.3 RASP-Generalization Conjecture

원 논문 기준 Zhou et al.의 주장은 theorem이 아니라 **phenomenological conjecture**다. 논문은 다음 세 조건을 만족할 때 decoder-only autoregressive Transformer가 length-generalize할 가능성이 높다고 제안한다.

1. **Realizability.** 하나의 decoder-only Transformer가 모든 입력 길이에 대해 true next-token function을 표현할 수 있다.
2. **Simplicity.** 그 표현이 RASP-L로 쓰일 만큼 단순하다.
3. **Diversity.** 훈련 분포에는 맞지만 OOD에서는 틀리는 더 짧은 RASP-L shortcut을 배제할 만큼 훈련 데이터가 다양하다.

이 conjecture의 논리를 최소한의 식으로 쓰면 다음과 같다. 훈련 데이터 D_N 에 맞는 프로그램

집합을

$$\mathcal{P}(D_N) = \{P : P(x) = f(x) \text{ for all } (x, f(x)) \in D_N\}$$

라 하자. simplicity bias를 이상화해서

$$\hat{P}_N \in \arg \min_{P \in \mathcal{P}(D_N)} \text{len}_{\text{RASP-L}}(P)$$

라고 생각한다. 진짜 알고리즘 P_* 가 짧고, data diversity가 모든 더 짧은 잘못된 shortcut P 를 제거하면

$$\hat{P}_N = P_*$$

가 가능하고, P_* 가 모든 길이에 정의되므로 length generalization이 따라온다는 그림이다.

주의 위의 $\arg \min$ 식은 Zhou et al.의 학습 알고리즘 정의가 아니라 conjecture를 이해하기 위한 이상화다. 논문도 실제 Transformer가 정확히 RASP-L program을 내부적으로 실행한다고 주장하지 않는다.

3.4 Scratchpad는 왜 어떤 때는 돕고 어떤 때는 방해하는가

고정-depth Transformer가 한 forward pass에서 순차 loop를 직접 반복하기는 어렵다. 하지만 autoregressive generation은

$$h_0 \rightarrow z_1 \rightarrow z_2 \rightarrow \dots$$

처럼 모델을 반복 호출하므로 바깥쪽 sequential loop를 제공한다. Scratchpad는 이 loop가 사용할 state를 text context에 외부화한다.

따라서 좋은 scratchpad의 기준은 “더 많은 중간 토큰”이 아니라

target task를 더 짧은 RASP-L next-token rule로 바꾸는가?

이다.

예를 들어 addition에서 carry를 멀리 전파해야 하는 표기보다 reverse-order output이나 index hint를 주어 carry를 causal/local하게 만들면 프로그램이 단순해진다. 반대로 scratchpad가 매 step마다 전역 통계를 복잡하게 다시 계산하게 만들면 program complexity가 오히려 증가할 수 있다.

3.5 RASP-L 관점의 장점과 한계

장점은 서로 다른 task를 하나의 질문으로 바꾼다는 것이다.

이 문제에 모든 길이에서 작동하는 짧은 Transformer-native program이 있는가?

하지만 세 가지를 기억해야 한다.

- RASP-L simplicity와 SGD implicit bias의 정확한 equivalence는 증명되지 않았다.
- shortest program을 실제로 찾는 문제 자체가 어려울 수 있다.
- “짧은 프로그램이 있다”는 것만으로 data diversity 조건까지 자동 충족되지는 않는다.

발표에서 이렇게 말하기 “Zhou 논문의 핵심은 길이 일반화를 parameter interpolation 문제가 아니라 program selection 문제로 바꾸는 것입니다. 모든 길이에 통하는 짧은 RASP-L 프로그램이 있고, 데이터가 더 짧은 shortcut을 제거하면 Transformer가 그 uniform algorithm을 선택할 가능성이 높다는 가설입니다. Scratchpad도 이 program을 짧게 만들 때만 도움이 됩니다.”

4

Huang et al.: 식별 가능성을 정리로 만들기

4.1 RASP-L conjecture에서 identifiability theorem으로

Zhou et al.의 그림에는 두 빈칸이 있다. 첫째, “simple”의 형식적 정의가 완전히 닫혀 있지 않다. 둘째, 짧은 입력을 점점 더 많이 관측하면 왜 잘못된 알고리즘이 결국 사라지는지 정리로 증명되지 않았다. Huang et al.은 이 문제를 **identification in the limit**로 바꾼다.

핵심 아이디어는 context size n 마다 별도의 finite Transformer T_n 을 생각하고, 이들이 커질 때 하나의 무한 길이 계산을 나타내는 Limit Transformer로 수렴하도록 구조를 제한하는 것이다.

4.2 Limit Transformer

표준 causal attention에서 layer l , head h 의 logit을

$$a_{i,j}^{(l,h)} = (y_j^{(l-1)})^\top K_{l,h}^\top Q_{l,h} y_i^{(l-1)}$$

라 하자. Limit Transformer는 여기에 위치 상호작용

$$\phi_{l,h}(j, i)$$

를 추가해

$$a_{i,j}^{(l,h)} = (y_j^{(l-1)})^\top K_{l,h}^\top Q_{l,h} y_i^{(l-1)} + \phi_{l,h}(j, i)$$

로 둔다.

이 객체는 실제로 학습하는 network가 아니라, context가 계속 커지는 Transformer sequence의 underlying algorithm을 기술하기 위한 수학적 limiting object다.

PERIODIC과 LOCAL

정의 4.1 (PERIODIC). 어떤 $\Delta > 0$ 가 존재하여 모든 i 에 대해

$$p_i = p_{i+\Delta}$$

이면 positional encoding이 PERIODIC하다고 한다.

정의 4.2 (LOCAL). causal index $j \leq i$ 에 대해 $\phi(j, i)$ 가 상대 거리 $i - j$ 에만 의존하고, 어떤 $\tau < \infty$ 가 존재하여

$$i - j > \tau \implies \phi(j, i) = 0$$

이면 LOCAL하다고 이해할 수 있다. 이는 원 논문의 translation-invariant/local 정의를 causal attention의 (j, i) argument 순서에 맞춰 다시 쓴 것이다.

PERIODIC은 무한히 많은 position parameter를 유한한 pattern으로 줄이고, LOCAL은 position-specific interaction이 무한히 먼 pair까지 독립적으로 늘어나는 것을 막는다.

4.3 Idealized inference procedure

context size n 에서 hypothesis class를 Θ_n 이라 하자. Huang et al.은 width 자체는 제한하지 않지만 parameter precision과 positional product의 translation invariance 등을 제한한다. 그리고 architecture/precision/norm/position interaction의 복잡도를 합친 regularizer $\mathcal{R}(T)$ 를 둔다.

목표 함수 f 에 대해

$$U_n = \left\{ T \in \Theta_n : T(x) = f(x) \text{ for all } |x| \leq \frac{n}{2} \right\}$$

라 하고,

$$\mathcal{R}(T_n) \leq \frac{1}{n} + \inf_{T \in U_n} \mathcal{R}(T)$$

를 만족하는 T_n 을 고른다.

즉 길이 $n/2$ 까지만 fitting하고 context n 까지 extrapolation하는 sequence를 만든다.

주의 이 inference procedure는 SGD의 정확한 모델이 아니다. norm-based regularizer를 거의 최소화하는 이상화된 선택 규칙이다. 따라서 다음 theorem을 “모든 실제 Transformer 훈련이 결국 length-generalize 한다”로 읽으면 안 된다.

4.4 Theorem 7: eventual length generalization

정리 4.1 (Huang et al., Theorem 7의 내용). f 에 대해 다음 두 조건은 동치다.

1. f 가 PERIODIC과 LOCAL을 만족하는 하나의 Limit Transformer로 모든 길이에서 표현된다.
2. 위 inference procedure가 생성하는 $T_1, T_2, \dots$ 에 대해

$$\sup_n \mathcal{R}(T_n) < \infty$$

이고, 어떤 N_0 가 존재하여 모든 $m > N_0$ 에서 T_m 이 길이 $k \leq m$ 인 모든 입력에서 f 와 일치한다.

훈련은 $m/2$ 까지만 맞추도록 요구했는데 결론은 m 까지 맞는다. 따라서 이 theorem이 의미하는 generalization factor는 표기상 2배이지만, 본질은 “training length가 충분히 커지면 finite threshold 이후에는 올바른 algorithm만 남는다”는 identification statement다.

4.5 왜 bounded regularizer가 중요한가

증명 직관을 단계별로 쓰면 다음과 같다.

1. PERIODIC이면 positional pattern은 $p_1, \dots, p_\Delta$ 로 압축된다.
2. LOCAL이면 positional interaction은 유한한 relative window $0, \dots, \tau$ 만 보면 된다.
3. bounded $\mathcal{R}$ 는 depth, head count, precision, matrix norm 등 algorithm description의 복잡도가 무한히 커지는 것을 막는다.
4. 결과적으로 bounded-complexity 영역에서 가능한 underlying discrete algorithms의 종류가 효과적으로 유한해진다.
5. 틀린 algorithm은 어떤 유한 길이에서 반드시 f 와 다른 출력을 낸다.
6. 후보가 유한하므로 모든 틀린 algorithm을 제거하는 최대 witness length도 유한하다.
7. 그 길이를 지난 뒤에는 모든 surviving hypothesis가 target과 모든 길이에서 같은 계산을 한다.

보충 유도 — 논문 밖의 독자용 전개 위 직관을 유한 후보 집합으로 단순화해 보자. 후보가

$$\mathcal{A}_C = \{A_1, \dots, A_M\}$$

이고 target을 A_* 라 하자. 각 잘못된 $A_r \neq A_*$ 에 대해 구별 witness 길이

$$n_r = \min\{n : \exists |x| \leq n, A_r(x) \neq A_*(x)\}$$

가 유한하다고 하자. 그러면

$$N_0 = \max_{A_r \neq A_*} n_r$$

도 유한하다. N_0 까지의 모든 입력에 맞는 후보는 더 이상 잘못된 A_r 일 수 없다. Huang et al.의 실제 증명은 bounded regularizer와 Limit Transformer 구조를 이용해 이 “finite algorithm” 논리를 정교하게 구현한다.

4.6 C-RASP와 lower/upper characterization

논문은 PERIODIC/LOCAL Limit Transformer의 expressivity를 C-RASP와 communication complexity로 분석한다. C-RASP[periodic, local]로 정의되는 프로그램은 해당 Limit Transformer로 옮길 수 있어 positive result를 준다. 반대로 이 class는 입력을 두 부분으로 나눈 communication 관점에서 제한을 갖기 때문에 모든 sequence function을 포함하지 않는다.

중요한 예시는 다음처럼 읽으면 된다.

- 일부 counting/majority 및 formal-language task는 framework 안에서 identifiability가 가능하다.
- repeated-token copy나 addition 같은 문제는 해당 restricted Limit Transformer class 밖에 놓일 수 있다.

이것은 Zhou et al.에서 scratchpad/index hint를 바꾼 addition이 잘 일반화한다는 관찰과 모순되지 않는다. **task representation**과 **autoregressive protocol** 자체를 바꾸면 계산해야 하는 함수가 달라지기 때문이다.

발표에서 이렇게 말하기 “Huang 논문은 RASP-L의 직관을 identification-in-the-limit 정리로 바꿉니다. PERIODIC과 LOCAL로 위치 계산의 자유도를 제한하고, bounded regularizer로 algorithm complexity를 묶으면 가능한 underlying algorithm이 결국 유한하게 압축됩니다. 짧은 길이가 충분히 길어지면 잘못된 후보는 모두 어느 유한 witness에서 제거되고 올바른 알고리즘만 남습니다.”

5

Izzo et al.: “충분히 길게”는 얼마나 긴가

5.1 Huang의 theorem이 남긴 정량적 질문

Huang et al.은 어떤 유한 N_0 가 존재함을 보이지만, N_0 가 실제로 얼마나 큰지는 알려 주지 않는다. Izzo et al.은 2026년에 다음 질문을 직접 다룬다.

핵심 두 Limit Transformer f, g 가 길이 N 이하의 모든 입력에서 거의 같은 동작을 보인다면, 어떤 N 이면 임의로 긴 입력에서도 반드시 거의 같다고 말할 수 있는가?

핵심 proof strategy는 **simulation**이다. 긴 문자열 x 가 주어졌을 때, 길이가 N 이하인 짧은 문자열 z 를 만들어

$$f(x) \approx f(z), \quad g(x) \approx g(z)$$

가 되게 한다. 이미 짧은 입력에서

$$f(z) \approx g(z)$$

임을 알고 있으므로 triangle inequality로

$$\begin{aligned} \|f(x) - g(x)\| &\leq \|f(x) - f(z)\| + \|f(z) - g(z)\| + \|g(z) - g(x)\| \\ &= O(\varepsilon) \end{aligned}$$

를 얻는다.

즉 정량적 length generalization의 질문은

긴 forward pass의 충분통계를 얼마나 짧은 sequence로 보존할 수 있는가?

로 바뀐다.

5.2 Finite precision에서 softmax가 hardmax가 되는 이유

한 layer Limit Transformer가 p -bit precision으로 계산된다고 하자. softmax 안정화를 위해 최대 logit을 뺀 후, exponentiated term이 2^{-p} 이하이면 0으로 round한다고 가정한다.

최대 logit과 비최대 logit 사이의 최소 양의 gap을 $\gamma(f) > 0$ 라 하자. sequence length를 N 이라 하면 logit에 $\log N$ scaling이 들어가므로 비최대 항의 exponent는

$$\exp(\log N (a - a_*)).$$

$a - a_* \leq -\gamma(f)$ 이므로

$$\exp(\log N (a - a_*)) \leq N^{-\gamma(f)}.$$

만약

$$N \geq 2^{p/\gamma(f)}$$

이면

$$N^{-\gamma(f)} \leq 2^{-p}.$$

따라서 모든 strictly submaximal term이 round-to-zero되고, softmax는 최대 logit을 얻는 위치들에 대한 uniform hardmax처럼 동작한다.

5.3 Theorem 4.1: finite-precision, one-layer bound

f, g 가 한 layer이고, τ -local, Δ -periodic, translation-invariant이며 p -bit precision에서 계산된다고 하자. 두 모델의 relevant norm scale을 합친 양을 L 이라 쓰고

$$\gamma = \min\{\gamma(f), \gamma(g)\}$$

라 하자.

정리 5.1 (Izzo et al., Theorem 4.1). 어떤

$$N = O\left(\max\left\{2^{p/\gamma}, \frac{L^2 \Delta^7 |\Sigma|^6 \tau^2}{\varepsilon^2}\right\}\right)$$

가 존재하여, 모든 $|x| \leq N$ 에서

$$\|f(x) - g(x)\| \leq \varepsilon$$

이면 임의 길이의 x 에 대해

$$\|f(x) - g(x)\| = O(\varepsilon)$$

이다.

이 식에서 읽어야 하는 것은 지수의 세부 숫자보다 dependence의 방향이다.

- L 이 크면 작은 입력 perturbation에 더 민감하므로 더 긴 훈련 길이가 필요하다.
- period Δ 가 크면 보존해야 할 position class가 많아진다.
- alphabet $|\Sigma|$ 가 크면 token/position empirical frequency의 종류가 많아진다.
- locality τ 가 커지면 유지해야 할 local pattern의 범위가 넓어진다.
- 더 작은 ε 을 원하면 N 이 증가한다.
- $2^{p/\gamma}$ 는 finite precision에서 hardmax regime에 진입하는 threshold다.

원 논문 기준 논문은 실제 hardmax behavior가 이 worst-case threshold보다 더 일찍 나타날 수 있다고 지적한다. 따라서 $2^{p/\gamma}$ 는 실험적인 “필수 길이의 정확한 값”이 아니라 theorem의 sufficient upper-bound 구성에서 나타나는 항이다.

5.4 왜 empirical frequency를 보존하면 되는가

hardmax regime에서는 각 query가 어떤 token/position class들의 argmax set에 attention하는 지가 유한한 유형으로 정리된다. 긴 sequence x 의 각 유형에 대한 비율을

$$\pi_c(x) = \frac{\#\{j : \text{position } j \text{ is class } c\}}{|x|}$$

라 하자. 짧은 z 가 모든 relevant class에 대해

$$|\pi_c(z) - \pi_c(x)| \leq \delta$$

를 만족하면 attention average도 가까워진다. 모델의 output Lipschitz scale이 L 이면 대략

$$\|f(x) - f(z)\| \lesssim L\delta.$$

따라서 $\delta \asymp \varepsilon/L$ 수준으로 histogram을 보존하는 짧은 sequence를 구성하면 된다. Theorem 4.1의 다항식 bound는 이 counting/simulation을 모든 class에 동시에 수행하면서 나온다.

5.5 Infinite precision: soft attention을 유지하면

finite precision에서는 긴 길이에서 attention이 hardmax로 단순화되지만, infinite precision에서는 작은 weight도 사라지지 않는다. Izzo et al.은 local suffix signal이 무한 길이에서 희석되지 않도록 suffix positional term에 $\log i$ 를 곱한다.

대략

$$a_{i,j} = \text{extcontentscore} + \log i \cdot \phi(j, i).$$

local positional exponent의 최대값에 따라 세 regime이 생긴다.

1. $\max \phi < 1$: token-dominant — global token frequency가 우세하다.
2. $\max \phi = 1$: balanced — local suffix와 global frequency가 같은 order로 남는다.
3. $\max \phi > 1$: position-dominant — local positional term이 지배한다.

두 layer infinite-precision class에서 모델 complexity를 $C(f)$, positional margin을 $\gamma(f)$ 라 두면 다음 형태의 quantitative guarantee를 얻는다.

정리 5.2 (Izzo et al., Theorem 5.2). 적절한 two-layer class $\mathcal{F}_\tau$ 의 f, g 에 대해

$$N \lesssim (\max\{C(f), C(g)\} \varepsilon^{-1})^{\max\{\gamma(f)^{-1}, \gamma(g)^{-1}, 3\}}$$

인 N 이 존재하여, 모든 $|x| \leq N$ 에서

$$\|f(x) - g(x)\| \leq \varepsilon$$

이면 임의의 길이에서

$$\|f(x) - g(x)\| = O(\varepsilon)$$

이다.

여기서 $C(f)$ 는 weight norm, τ , $|\Sigma|$ 등을 포함하는 논문 특유의 complexity measure다. 정확한 식은 첫 layer attention의 Lipschitz constant 때문에 weight norm에 대해 exponential dependence를 포함할 수 있다.

5.6 Huang과 Izzo를 한 문장으로 연결하면

Huang:

bounded-complexity algorithm class $\implies \exists N_0$ after which identification succeeds.

Izzo:

simulation complexity를 계산 $\implies N_0$ 의 sufficient quantitative upper bound.

따라서 2026 결과는 단순한 “더 강한 theorem”이라기보다 identification argument를 non-asymptotic하게 만든 것이다.

발표에서 이렇게 말하기 “Huang은 충분히 긴 길이에서 잘못된 알고리즘이 결국 제거된다고 보였지만 그 길이가 얼마인지는 말하지 않았습니다. Izzo는 긴 forward pass를 짧은 sequence로 simulation하는 문제로 바꾸고, period, locality, vocabulary size, weight norm, margin과 목표 오차가 필요한 training length를 어떻게 키우는지 정량적으로 보입니다.”

6

Algorithmic generalization을 동역학으로 보기

6.1 같은 transition rule을 더 오래 반복한다

많은 algorithmic task는 내부 state를

$$q_{t+1} = F(q_t, x_t)$$

로 업데이트하는 순차 계산으로 쓸 수 있다. 훈련에서 짧은 sequence만 봤다는 것은 transition을 적은 횟수만 반복해 본 것이고, length generalization은 같은 rule을 훨씬 더 많이 반복했을 때도 trajectory가 올바른지를 묻는다.

Transformer가 이를 한 layer 안에서 직접 recurrence로 실행하지 않더라도, scratchpad/autoregressive decoding을 사용하면

$$q_t \rightsquigarrow z_t$$

처럼 state를 token sequence에 기록하고 같은 next-step map을 반복 적용하는 형태가 된다.

6.2 Local step error가 길이에 따라 어떻게 누적되는가

보충 유도 — 논문 밖의 독자용 전개 이 절의 유도는 특정 논문의 theorem이 아니라, 왜 “짧은 길이에서 높은 step accuracy”만으로 긴 길이를 보장할 수 없는지 설명하는 표준 안정성 계산이다.

진짜 dynamics와 학습된 dynamics를

$$q_{t+1} = F(q_t, x_t), \quad \hat{q}_{t+1} = \hat{F}(\hat{q}_t, x_t)$$

라 하자. F 가 state에 대해 L_F -Lipschitz이고

$$\|\hat{F}(q, x) - F(q, x)\| \leq \varepsilon$$

라고 하자. 오차를

$$e_t = \|\hat{q}_t - q_t\|$$

라 하면

$$\begin{aligned} e_{t+1} &= \|\hat{F}(\hat{q}_t, x_t) - F(q_t, x_t)\| \\ &\leq \|\hat{F}(\hat{q}_t, x_t) - F(\hat{q}_t, x_t)\| + \|F(\hat{q}_t, x_t) - F(q_t, x_t)\| \\ &\leq \varepsilon + L_F e_t. \end{aligned}$$

$e_0 = 0$ 이면 반복해서

$$e_T \leq \varepsilon \sum_{j=0}^{T-1} L_F^j.$$

따라서

$$e_T \leq \begin{cases} \varepsilon \frac{1 - L_F^T}{1 - L_F}, & L_F < 1, \\ T\varepsilon, & L_F = 1, \\ \varepsilon \frac{L_F^T - 1}{L_F - 1}, & L_F > 1. \end{cases}$$

해석은 명확하다.

- $L_F < 1$: contractive transition이면 error가 bounded하게 유지될 수 있다.
- $L_F = 1$: 작은 local error가 길이에 선형으로 쌓인다.
- $L_F > 1$: 작은 mismatch도 긴 horizon에서 지수적으로 증폭될 수 있다.

이 때문에 algorithmic generalization은 단순한 “one-step imitation”이 아니라 **iterated computation**의 **stability** 문제이기도 하다.

6.3 Scratchpad와 locality

좋은 scratchpad는 global computation을 local transition으로 바꾼다. 예를 들어 전체 결과를 한 번에 계산하는 함수

$$y = G(x_{1:n})$$

대신

$$q_{t+1} = F(q_t, x_t), \quad y = H(q_n)$$

로 문제를 표현하고 각 q_t 를 scratchpad에 쓰면, 모델이 학습해야 할 next-token rule의 complexity가 줄 수 있다.

그러나 state 표현이 길이에 따라 새 index arithmetic을 요구하거나, 매 step마다 전체 prefix를 다시 count해야 한다면 scratchpad가 오히려 RASP-L complexity를 높인다. 즉 “중간 reasoning을 보인다”는 사실 자체보다 중간 **representation**이 계산을 **local/uniform**하게 만드는가가 중요하다.

6.4 Fixed depth와 autoregressive depth의 차이

한 번의 Transformer forward pass depth를 L 이라 하자. L 은 입력 길이와 무관하게 고정된다. 그런데 T 개의 scratchpad token을 생성하면 같은 block을 T 번 호출하므로 effective sequential depth가 증가한다.

$$\text{one-shot: } O(L) \quad \text{vs.} \quad \text{autoregressive scratchpad: } O(LT).$$

이 관점은 Week 5의 computational-theory 결과와 연결된다. 하지만 여기서는 “계산 가능성”보다 짧은 **training trajectory**에서 배운 **transition**이 더 긴 **horizon**으로 안정적으로 반복되는가가 중심이다.

발표에서 이렇게 말하기 “알고리즘 길이 일반화를 recurrence 관점에서 보면, 짧은 sequence에서 배운 한-step transition을 더 오래 반복하는 문제입니다. 그래서 local step error가 얼마나 누적되는지가 중요하고, 좋은 scratchpad는 복잡한 global computation을 짧고 안정적인 local transition으로 바꿉니다.”

7

Turing Programs: 모든 알고리즘에 같은 형식을 줄 수 있는가

7.1 RASP-L simplicity의 task dependence

Zhou et al.의 접근에서 task마다 좋은 scratchpad를 사람이 설계해야 할 수 있다. Addition의 좋은 scratchpad와 multiplication의 좋은 scratchpad는 다를 수 있다. Hou et al.은 이 문제를 더 보편적인 질문으로 바꾼다.

임의의 알고리즘을 동일한 local next-step 형식으로 바꿀 수 있는가?

그들이 제안하는 **Turing Program**은 computation을 Turing machine의 한 step과 유사한 textual state update로 분해한다. 각 생성 step은 대부분 context를 복사하고 head/state 주변의 소수 symbol만 수정하도록 설계된다.

7.2 왜 이 형식이 length generalization에 유리한가

일반 알고리즘 A 가 입력 길이에 따라 실행 step 수가 증가한다고 하자. 직접 답을 내는 format에서는 model이

$$x \mapsto A(x)$$

라는 전체 계산을 한 번에 배워야 한다. Turing Program에서는

$$s_{t+1} = \delta(s_t)$$

라는 동일한 local transition을 반복한다.

훈련에서 짧은 computation만 봐도 transition δ 자체가 length-independent하면, 더 긴 test에서는 같은 rule을 더 많이 적용하는 것으로 바뀐다.

Hou et al.은 addition, multiplication, in-context SGD 등의 algorithmic task에서 이 형식의 length generalization을 실험하고, 이론적으로 임의의 Turing machine을 simulate하는 simple RASP program을 구성해 Transformer의 representability를 보인다.

주의 “Turing Programs가 universal하다”는 말은 임의의 computable algorithm을 이 형식으로 표현/시뮬레이션할 수 있다는 의미다. 이 사실만으로 finite data와 SGD가 모든 임의 Turing Program의 transition을 항상 학습한다는 universal learnability theorem이 되는 것은 아니다. 논문의 이론적 construction과 empirical generalization claim을 구분해야 한다.

7.3 Week 7의 CoT와 연결

Week 7에서는 CoT가 sequential computation budget을 늘린다고 보았다. Week 8에서는 한 단계 더 나아가 어떤 CoT representation이 길이 외삽 가능한 transition rule을 만드는가를 묻는다.

일반 CoT:

$$z_{t+1} = F_{\theta}(x, z_{1:t})$$

는 매 step 계산이 전역적일 수 있다.

Turing-style representation:

$$s_{t+1} = \delta_{\theta}(s_t)$$

는 가능한 한 local하고 time-homogeneous하게 만든다. 따라서 reasoning-token 수는 늘어나지만 next-step function의 algorithmic complexity는 줄어드는 tradeoff가 생긴다.

발표에서 이렇게 말하기 “Turing Programs의 아이디어는 task마다 특별한 scratchpad를 찾는 대신, 모든 알고리즘을 Turing-machine-like local transition으로 바꾸자는 것입니다. 이론적으로는 임의 Turing machine을 RASP로 simulate할 수 있고, 실험적으로는 여러 알고리즘에서 이 representation이 길이 일반화를 크게 개선합니다. 다만 representability가 곧 universal SGD guarantee라는 뜻은 아닙니다.”

8

통합 관점: 무엇이 길이 일반화를 결정하는가

8.1 네 논문의 역할을 한 줄씩

논문	질문	답의 형태
Kazemnejad et al. (2023)	PE가 길이 외삽에 어떤 영향을 주는가?	empirical comparison + NoPE의 position representability construction
Zhou et al. (2024)	어떤 task에서 Transformer가 알고리즘을 외삽하는가?	short RASP-L program에 대한 phenomenological conjecture
Huang et al. (2025)	이 직관을 formal identifiability로 만들 수 있는가?	PERIODIC/LOCAL Limit Transformer + bounded regularizer의 eventual LG theorem
Izzo et al. (2026)	“충분히 긴 training length”가 얼마나 긴가?	simulation argument를 이용한 quantitative upper bounds
Hou et al. (2025)	임의 알고리즘에 보편적인 LG-friendly format이 있는가?	Turing Programs + RASP simulation construction

8.2 Length generalization의 네 조건

이번 주의 결과를 하나의 체크리스트로 압축하면 다음 네 축이 반복해서 등장한다.

1. Uniform realizability

하나의 고정 규칙이 모든 길이에서 정의되어야 한다.

$$\exists A_* \quad \forall n, \quad A_*|_{\Sigma^n} = f|_{\Sigma^n}.$$

길이마다 다른 lookup table을 필요로 한다면 진정한 algorithmic extrapolation이라고 보기 어렵다.

2. Simplicity / bounded complexity

올바른 알고리즘이 Transformer-native하게 단순해야 한다. Zhou에서는 짧은 RASP-L, Huang에서는 bounded regularizer, Izzo에서는 bounded norm/locality/periodicity가 이에 해당한다.

3. Identifiability / diversity

짧은 training data가 잘못된 shortcut을 제거할 만큼 다양해야 한다.

$$P \neq P_* \quad \Rightarrow \quad \exists x \text{ in observed range} : P(x) \neq P_*(x).$$

이 조건이 없으면 training error 0인 shortcut이 남는다.

4. Stable execution

알고리즘을 더 긴 horizon에서 반복했을 때 작은 representation/error가 폭발하지 않아야 한다. Positional score dilution, transition error accumulation, attention margin이 모두 이 축과 관련된다.

8.3 서로 모순처럼 보이는 결과를 정리하기

“NoPE도 position을 표현한다” vs. “position restriction이 필요하다”. 모순이 아니다. 유한 context에서 position을 표현할 수 있다는 것과 bounded complexity로 무한 길이에 uniform하게 extension된다는 것은 다르다.

“Addition은 RASP-L format을 바꾸면 일반화한다” vs. “Huang class에서는 addition이 제외될 수 있다”. input/output/scratchpad protocol이 다르면 target sequence function 자체가 다르다. Autoregressive intermediate state가 계산 자원을 바꾼다.

Turing completeness와 length generalization. Turing-completeness는 적절한 weight, precision, protocol이 존재한다는 expressivity statement다. 짧은 데이터와 SGD가 그 weight를 선택한다는 identifiability/learnability statement와 무관하다.

“eventually generalizes” vs. “실제로는 길이가 조금만 늘어도 실패한다”. Asymptotic existence threshold N_0 가 실용적인 크기일 필요는 없다. Izzo의 bound가 바로 complexity에 따라 required N 이 크게 증가할 수 있음을 보여 준다.

8.4 발표에서 꼭 구분할 네 단어

용어	의미
Representable	어떤 parameter가 모든 길이에 목표 함수를 구현할 수 있다.
Identifiable	충분한 짧은 관측을 보면 잘못된 후보를 제거할 수 있다.
Learnable	실제 optimization procedure가 좋은 후보를 찾는다.
Stable	더 긴 execution에서도 작은 local mismatch가 제어된다.

발표에서 이렇게 말하기 “이번 주 전체를 한 문장으로 요약하면, length generalization은 ‘긴 위치를 encode할 수 있느냐’보다 ‘짧은 데이터에서 모든 길이에 통하는 단순하고 안정적인 프로그램을 식별할 수 있느냐’의 문제입니다. Zhou가 simplicity 가설을 주고, Huang이 asymptotic identifiability를 증명하고, Izzo가 필요한 training length를 complexity의 함수로 정량화합니다.”

9

발표용 정리와 예상 질문

9.1 칠판에 남겨야 할 핵심 식

1. Length generalization setting

$$|x| \leq N_{\text{tr}} \text{에서 학습} \implies |x| > N_{\text{tr}} \text{에서 같은 알고리즘}$$

2. Finite-length non-identifiability

$$\tilde{f}_N(x) = \begin{cases} f(x), & |x| \leq N, \\ g(x), & |x| > N. \end{cases}$$

3. NoPE causal position signal

$$\alpha_{tj} = \frac{1}{t} \implies \text{BOS value output} = \frac{1}{t}.$$

4. Huang inference rule

$$\mathcal{R}(T_n) \leq \frac{1}{n} + \inf_{T \in U_n} \mathcal{R}(T), \quad U_n = \{T : T = f \text{ on } |x| \leq n/2\}.$$

5. Finite-precision hardmax threshold

$$N \geq 2^{p/\gamma} \implies N^{-\gamma} \leq 2^{-p}.$$

6. Izzo finite-precision training-length bound

$$N = O\left(\max\left\{2^{p/\gamma}, \frac{L^2 \Delta^7 |\Sigma|^6 \tau^2}{\varepsilon^2}\right\}\right).$$

7. Transition-error accumulation

$$e_{t+1} \leq L_F e_t + \varepsilon \implies e_T \leq \varepsilon \sum_{j=0}^{T-1} L_F^j.$$

8. Unifying simulation inequality

$$\|f(x) - g(x)\| \leq \|f(x) - f(z)\| + \text{orm}f(z) - g(z) + \text{orm}g(z) - g(x).$$

9.2 예상 질문 1: “그러면 NoPE가 가장 좋은 PE인가?”

아니다. Kazemnejad et al.의 특정 실험에서는 NoPE가 여러 explicit PE보다 강했지만, 이는 architecture/task/training distribution에 의존하는 empirical result다. 이론적 결과는 NoPE가 causal mask를 이용해 absolute/relative position을 표현할 수 있음을 보일 뿐이다.

9.3 예상 질문 2: “RASP-L conjecture는 증명된 것인가?”

아니다. Zhou et al.은 이를 phenomenological conjecture라고 명확히 부른다. Huang et al.은 related intuition을 엄밀화하지만, 실제 SGD 전체에 대한 RASP-L conjecture를 그대로 증명한 것은 아니다. 그들의 guarantee는 PERIODIC/LOCAL Limit Transformer와 특정 idealized inference procedure 아래에서 성립한다.

9.4 예상 질문 3: “Theorem 7이면 충분히 오래 학습하면 항상 일반화하는 것 아닌가?”

아니다. target이 해당 Limit Transformer class로 표현되어야 하고, inference sequence의 regularizer가 bounded해야 한다. 또 실제 training optimizer와 동일한 procedure가 아니다. 더구나 “충분히 길다”의 threshold가 실용적으로 작다는 보장도 Theorem 7만으로는 없다.

9.5 예상 질문 4: “Izzo bound는 necessary condition인가?”

아니다. 논문은 sufficient upper bound를 준다. 실제 task/model은 worst-case bound보다 훨씬 짧은 길이에서 generalize할 수 있다. 특히 finite-precision hardmax threshold $2^{p/\gamma}$ 도 sufficient analysis에서 나타나는 항이다.

9.6 예상 질문 5: “RoPE나 ALiBi가 있으면 무한 길이에 외삽 가능한가?”

위치 함수 자체는 새로운 index에 계산될 수 있지만, 전체 network computation이 목표 알고리즘을 유지한다는 보장은 아니다. attention dilution, unseen relative-distance regime, learned content interaction 등의 문제는 여전히 남는다.

9.7 예상 질문 6: “Turing Programs와 Turing completeness는 같은 말인가?”

관련 있지만 다르다. Turing completeness는 모델 계산계의 표현력에 관한 statement다. Turing Programs는 임의 algorithm을 local textual transition으로 바꿔 Transformer가 학습하기 쉬운 length-generalizing format을 만들려는 방법이다. Hou et al.은 임의 Turing machine을 simulate하는 RASP construction을 제공하지만, finite-data SGD에 대한 universal learnability까지 뜻하지는 않는다.

9.8 마지막 3분 요약

발표 마지막에는 다음 순서로 정리하면 흐름이 자연스럽다.

1. **Position:** PE가 길이 바깥에 정의되는 것과 알고리즘이 외삽되는 것은 다르다.
2. **Program:** Zhou et al.은 모든 길이에 통하는 짧은 RASP-L program을 Transformer-native simplicity로 제안한다.
3. **Identification:** Huang et al.은 PERIODIC/LOCAL + bounded regularizer 아래에서 eventual identification을 증명한다.
4. **Quantification:** Izzo et al.은 긴 computation을 짧은 sequence로 simulate하여 required training length를 parameter complexity의 함수로 묶는다.

5. **Representation:** Scratchpad/Turing Program은 hard global algorithm을 반복 가능한 local transition으로 바꿀 수 있다.

핵심 이번 주의 가장 중요한 구분은

$\text{expressivity} \neq \text{identifiability} \neq \text{learnability} \neq \text{stability}$

이다. “Transformer가 이 알고리즘을 표현할 수 있다”에서 “짧은 예제로 학습한 모델이 더 긴 길이에 일반화한다”까지 가려면 이 네 단계가 모두 필요하다.

발표에서 이렇게 말하기 “Length generalization을 단순히 positional encoding 문제로 보면 설명이 부족합니다. 진짜 문제는 짧은 데이터가 모든 길이에 통하는 uniform program을 식별하게 만드는 inductive bias가 무엇인지입니다. 프로그램의 단순성, positional locality와 periodicity, 충분한 training diversity, 그리고 긴 horizon에서의 안정성이 함께 맞아야 합니다.”

참고문헌

1. Amirhossein Kazemnejad, Inkit Padhi, Karthikeyan Natesan Ramamurthy, Payel Das, and Siva Reddy. **The Impact of Positional Encoding on Length Generalization in Transformers**. NeurIPS, 2023.
2. Hattie Zhou, Arwen Bradley, Etai Littwin, Noam Razin, Omid Saremi, Joshua Susskind, Samy Bengio, and Preetum Nakkiran. **What Algorithms Can Transformers Learn? A Study in Length Generalization**. ICLR, 2024.
3. Xinting Huang, Andy Yang, Satwik Bhattamishra, Yash Sarrof, Andreas Krebs, Hattie Zhou, Preetum Nakkiran, and Michael Hahn. **A Formal Framework for Understanding Length Generalization in Transformers**. ICLR, 2025.
4. Zachary Izzo, Eshaan Nichani, and Jason D. Lee. **Quantitative Bounds for Length Generalization in Transformers**. ICLR, 2026.
5. Kaiying Hou, David Brandfonbrener, Sham M. Kakade, Samy Jelassi, and Eran Malach. **Universal Length Generalization with Turing Programs**. ICML, 2025.
6. Gail Weiss, Yoav Goldberg, and Eran Yahav. **Thinking Like Transformers**. ICML, 2021.